

哈爾濱工業大學

本科毕业论文（设计）开题报告

题 目：基于可视化与交互式模拟的 LSM-Tree 存储
结构性能分析与优化展示系统设计

专 业 计算机科学与技术

学 生 刘新雨

学 号 2022111225

指导教师 苗东菁

日 期 2025/10/23

目录

1.课题来源及研究的目的和意义	3
1.1 课题来源	3
1.2 研究目的	3
1.3 研究意义	4
2.国内外在该方向的研究现状及分析	4
2.1 历史与产业背景（起源与工业实践）	4
2.2 研究主流方向概览	4
2.3 近年代表性工作与其贡献	5
2.4 国内研究与工程现状	6
2.5 存在的问题与研究空间	7
3.主要研究内容	8
3.1 LSM-TREE 核心机制实现	8
3.2 性能采集与事件追踪机制	8
3.3 前端交互式可视化展示	9
3.4 系统封装与部署	10
4.研究方案	10
4.1 系统架构设计	10
5.进度安排和预期达到的目标	11
5.1 进度安排	11
5.2 预期目标	12
6.课题已具备和所需的条件、经费	12
7.研究过程中可能遇到的困难和问题，解决的措施	12
8.主要参考文献	13

1.课题来源及研究的目的和意义

1.1 课题来源

在大数据、云计算、分布式存储迅速发展的背景下，海量数据的高写入吞吐、高并发处理、低延迟访问成为了存储系统的重要挑战。在这一背景下，基于写优化

(write-optimized) 思想的 Log-Structured Merge Tree (LSM-Tree) 结构因其利用内存缓冲、顺序写入、后台 compaction 合并的特点，逐渐成为现代 NoSQL / KV 存储系统的主流选择。典型产品如 LevelDB、RocksDB、HBase 等均在其底层或索引结构中采用了 LSM-Tree[1]。

然而，尽管应用广泛，LSM-Tree 的内部机制（如 MemTable、SSTable、flush、compaction、Bloom Filter、并发线程、策略切换等）结构复杂，对于教学、初学者理解以及存储系统研究人员进行性能分析与优化仍然具有较高的理解门槛。基于此，本课题拟设计一个面向教学、科研目的的“可视化与交互式模拟”系统，以直观展示 LSM-Tree 的内部流程与性能特性，从而起到教学辅助与研究支持的作用。

1.2 研究目的

本课题的主要目的可从四个核心维度展开，通过整合功能设计与用户需求，实现多场景下的价值输出。首先是深化理解层面，系统将面向高校相关课程学生、存储系统初学者及系统研究人员三类群体，借助可视化与交互式界面，帮助他们全面掌握 LSM-Tree 的关键技术细节，包括其工作原理、数据流转机制、内存与磁盘的交互逻辑、并发线程的状态变化，以及不同 compaction 策略之间的性能差异，让抽象的技术原理转化为直观可感知的内容。

在此基础上，系统将承担教学与科研辅助的角色。对于教师，可将其作为课堂演示工具，清晰呈现 LSM-Tree 的运行过程；对于学生，支持通过交互操作调整参数，实时观察参数变化带来的效果，强化实践认知；对于研究人员，则能借助系统演示 size-tiered 与 leveled 等不同策略下的 LSM-Tree 行为，为性能分析提供直观的实验平台。

同时，系统还将聚焦性能分析与优化辅助功能，通过内置性能监控模块，实时采集写放大、延迟、I/O 次数等关键指标，并以图表和动画的形式呈现数据，帮助用户快速定位性能瓶颈，直观对比不同优化方案的效果。

最后，在技术实现层面，系统将采用 Web 前端与后端事件追踪相结合的当前主流前后端架构，并通过 Docker 等容器化技术进行封装部署，最终实现“一键运行”的便捷操作和高可移植性，确保其能轻松适配教学机、研究服务器、云平台等不同使用环境，降低部署门槛。

1.3 研究意义

本课题的研究意义可从理论、应用与社会三个层面逐层递进，形成完整的价值体系。在理论意义层面，课题聚焦 LSM-Tree 的抽象机制与具体运行行为的深度结合，通过可视化与交互模拟技术，将传统文本或静态图示难以直观呈现的动态过程（如数据流转、compaction 执行）转化为可交互的视觉内容，不仅填补了当前存储结构领域教学工具的空白，也为存储系统研究人员搭建了便捷的调试与可视化平台，助力其更高效地探索 compaction 策略优化、线程调度逻辑、I/O 行为分析等关键方向，推动 LSM-Tree 相关理论研究的深化。

从应用价值来看，该系统可实现多场景落地。在高校本科与研究生的存储系统相关课程中，教学者借助系统直观演示 LSM-Tree 的运行逻辑，能将抽象技术原理转化为可感知的操作体验，显著提升学生的理解效率与学习兴趣；而在科研或工业存储系统研发场景中，开发者可通过系统快速验证不同策略对性能的影响，无需搭建复杂的实体环境即可完成参数调优与方案预研，有效缩短研发周期、降低设计成本。

这一研究的价值进一步延伸至社会层面。当前数据规模持续扩大，物联网、日志系统、实时流处理等高并发写入场景日益普遍，LSM-Tree 作为核心存储结构，其优化研究对数字基础设施发展具有重要意义。本系统通过降低先进存储结构的认知门槛，帮助高校学生与研发人员快速提升相关领域的认知水平与实践能力，最终将为我国数据库与存储系统领域的教育质量提升、研究创新突破及工程实践优化提供有力支撑。

2.国内外在该方向的研究现状及分析

2.1 历史与产业背景（起源与工业实践）

LSM-Tree 最早由 O'Neil 等人在 1996 年提出[2]，作为一种为高频插入场景设计的磁盘索引结构，通过将写操作先缓存在内存、再批量地顺序写入磁盘并在后台合并（merge/compaction），实现了对高写入吞吐场景的优良支持。该工作奠定了 LSM-Tree 理论基础。

随后，Google 等工业界把 LSM-Tree 的思想用于分布式存储系统[3]（如 Bigtable），推动了 LSM-Tree 在生产系统中的广泛采用；基于 LSM 的开源实现（LevelDB、RocksDB 等）成为众多工程系统的核心组件。以上工作把 LSM-Tree 从理论带入大规模工程实践，成为当前许多 NoSQL/KV 存储的基础。

2.2 研究主流方向概览

近年来，学术界与工业界对 LSM-Tree 的研究已形成多维度、成体系的探索方向，各方向均积累了大量学术成果与工程实践经验。这些研究首先聚焦于 LSM-Tree 的核心机制——compaction 与层次结构设计，核心目标是通过优化分层逻辑与合并策略减少写放大、降低访问延迟或平衡读写性能，典型如 size-tiered 与 leveled 两种策略的对比优化，同时也涵盖 compaction 调度逻辑与参数调谐的理论方法，例如 Monkey 提出的最优 Bloom 过滤器与层大小联合调优方案 [4]。

其次是索引与过滤结构的性能优化，该方向以提升查询效率为核心，主要通过在各级 SSTable 部署 Bloom 过滤器快速过滤不存在的查找请求，并探索有限内存下的过滤器资源分配策略，如 Monkey 提出的按级别不均匀分配方案以降低总体成本；此外，各类替代或改进型过滤器（如 cuckoo filter、quotient filter 等）的研发也成为重要分支 [5]。

与此同时，面向实际应用场景的工作负载自适应与参数调优研究逐渐成为重点，该方向针对不确定或动态变化的工作负载，通过 robust tuning 技术（如 Endure）与运行时自适应调整（包括动态修改层比、合并阈值、布隆过滤器分配等），提升 LSM-Tree 在复杂场景下的性能稳健性 [6]。进一步地，新硬件适配与数据布局设计研究则围绕 SSD、SMR、NVM 等新型存储介质的特性展开，通过改造 LSM-Tree 架构适配硬件特性，例如 WiscKey 针对 SSD 设计的 key/value 分离方案，以及面向 SMR 的优化系统 SEALDB [7]。

此外，技术融合与范式创新方向也涌现出诸多探索，典型如将 learned index（学习型索引）思想引入 LSM-Tree（如 Bourbon 系统 [8]），通过模型近似索引结构减少查找成本，或优化 SSTable 内部数据组织形式。最后，系统测量、基准测试与跟踪工具的研发是支撑上述所有研究的基础，该方向通过对 RocksDB、LevelDB 等主流 LSM-Tree 实现进行实测与建模 [9]，深入解析真实工作负载下的系统行为，为各类优化方案提供验证依据与方向指导。

2.3 近年代表性工作与其贡献

为清晰呈现 LSM-Tree 领域的研究脉络，本节挑选若干具有奠基意义、高工业影响力或前沿探索价值的代表性工作，系统分析其核心贡献，并明确其与本课题的关联点，为后续系统设计提供理论与实践参考。

首先是 O'Neil 等人于 1996 年提出的 “The Log-Structured Merge-Tree” [2]，该工作首次明确了 LSM-Tree 的基本设计思想与核心算法，从理论层面定义了 MemTable、SSTable、compaction 等关键组件的运行逻辑，为整个领域奠定了理论基础。其与本课题的关联在于，可为系统中 “LSM-Tree 原理讲解” 模块提供核心依据，帮助用户理解各组件的工作机制及性能权衡逻辑。

紧随其后，Chang 等人在 2006 年 OSDI 会议上提出的 Bigtable [3]，将 LSM-Tree 思想成功应用于大规模分布式存储场景，完成了从理论到工业级实践的关键跨越。该工作不仅验证了 LSM-Tree 在海量数据存储中的价值，还深入探索了分布式环境下的容错、扩展等工程化问题，其定义的性能度量指标也成为行业标准。对本课题而言，Bigtable 的工业实现细节可为本系统的 “性能指标监控” 模块提供参考，确保监控维度与工业实践对齐。

作为 LSM-Tree 开源生态的核心推动力量，Google 的 LevelDB 与 Facebook 的 RocksDB 及其后续研究具有不可替代的价值。LevelDB 以轻量级设计降低了 LSM-Tree 的使用门槛，推动了开源生态的发展；RocksDB 则在生产场景的长期演进中，积累了大量参数调优、工程优化方案 [10]，而围绕 RocksDB 的工作负载刻画、优化优先级研究，更成为理解真实系统行为的关键依据。这些工作与本课题直接相关，可作为演示系统的参考实现与实验基线，确保系统演示效果符合工业实际。

在硬件适配与数据布局优化方向，Lu 等人于 2016 年 FAST 会议提出的 WiscKey [11] 极具代表性。该工作创新性地采用 key/value 分离架构，针对性解决 SSD 存储介质下的写放大问题，通过优化数据布局显著提升了存储效率。其与本课题的关联在于，可作为“数据布局对 I/O 放大影响”的典型案例，融入系统的可视化与对比实验模块，帮助用户直观理解硬件特性与架构设计的适配逻辑。

在 compaction 策略与过滤结构优化领域，Dayan 等人 2017 年 SIGMOD 会议提出的 Monkey [4] 贡献突出。该工作首次提出在给定内存预算下，对 Bloom filter 与层大小进行联合最优分配的理论模型与实践方法，为后续相关研究提供了核心思路。对本课题而言，Monkey 的优化逻辑可作为“策略对比”与“参数调优”模块的理论依据，支持系统复现其关键实验，帮助用户理解资源约束下的优化策略设计。

同样聚焦性能优化的还有 Raju 等人 2017 年 SOSP 会议提出的 PebblesDB [12]，以及 Ren 等人在 SIGMOD/FAST 等会议上发表的 SlimDB [13]。这些工作围绕“碎片化治理”与“空间高效利用”展开，通过多项工程创新实现了“降低写放大”与“保持读写吞吐”的平衡。它们与本课题的关联在于，可作为“不同实现方案对性能影响”的候选案例，融入可视化模块，让用户直观观察不同架构设计如何影响写放大与访问延迟。

在技术范式创新层面，2020 年 OSDI 会议提出的 Bourbon [8] (From WiscKey to Bourbon) 展现了跨界融合的价值。该工作将 learned index (学习型索引) 思想引入 LSM-Tree，通过模型近似索引结构减少查找开销，验证了机器学习思想在存储结构优化中的可行性。其与本课题的关联在于，可作为“新型索引 / 增强方案”的核心展示对象，丰富系统的技术覆盖维度，帮助用户了解领域前沿趋势。

而在前沿探索方向，2024 年提出的 Endure [6] 等工作则聚焦“稳健调优与扩展性”，针对工作负载不确定或动态变化的场景，研究鲁棒性参数选择方法，凸显了运行时自适应优化的研究价值。这类工作虽暂未直接融入当前系统设计，但为课题的未来扩展提供了方向——可将“自适应参数调整的动态可视化”作为后续功能迭代重点，进一步提升系统的前沿性与实用性。

2.4 国内研究与工程现状

在工程实践方面，国内多家数据库厂商已在其核心存储引擎中采用或改进了 LSM-Tree 结构。例如，OceanBase 的存储引擎基于 LSM-Tree 架构，将数据分为增量 (MemTable) 与基线 (SSTable) 两部分，并结合定期 Major Compaction 与三层结构进行优化 [14]。本文中提到的该系统“基于 LSM-tree 的存储系统”已在金融核心场景通过检验。此外，OceanBase 的存储引擎官方博客也指出其将 LSM 树架构与自研存储强化整合，有别于直接使用开源版本。这些企业级实践表明，LSM-Tree 已成为国产分布式数据库的主流底层存储方案之一。

在学术研究方面，国内高校和科研机构围绕 LSM-Tree 的结构优化、硬件适配 (如 NVM/SSD)、系统调度等方向展开研究。例如，华东师范大学团队在《优化基于非易失性存储器的 LSM-Tree 存储系统》[15] 一文中针对 NVM 引入了新的设计方法，以缓解写放大、写暂停与读不友好问题。虽然国内专门以“LSM-Tree 可视化工具”命名的研究相对少见，但公开文献已明确探讨了 LSM-Tree 在中国环境下的优化可能性。此外，OceanBase

存储技术报告归纳了其基于 LSM-Tree 存储系统的架构与关键技术，进一步体现了工程研究与学术思考的结合。

总体来看，国内在 LSM-Tree 方向的研究正在从单一算法优化向系统工程化与可视化教学工具化方向拓展。然而，现有研究与工程实现仍主要集中在存储引擎性能提升与硬件适配方面，而针对 LSM-Tree 的可视化展示、交互式模拟与性能对比平台的系统性研究仍显稀缺。这也为本课题提供了重要的研究空间和创新方向。

2.5 存在的问题与研究空间

通过对上述文献与工程工作的系统梳理，可从现有研究与实践中提炼出四类关键问题，这些问题既反映了当前领域的需求缺口，也为本课题明确了核心切入方向。

首先是可视化与教学工具的显著短缺。当前 LSM-Tree 领域虽积累了大量理论突破与工程优化成果，但多聚焦于算法设计、系统实现与性能评测，面向教学场景与交互式可视化的工具极为有限。现有工作普遍缺乏对系统运行时动态过程的直观呈现能力，例如 MemTable 写入、SSTable flush、compaction 执行等关键事件，以及线程状态、I/O 行为、内存与磁盘占用变化等细粒度信息，均难以通过现有工具具象化——这一缺口恰好为本课题提供了核心方向，即通过构建交互式演示系统，补齐“教学 + 研究”双场景下的可视化工具空白。

其次是真实工作负载的可观测性不足。在生产环境中，获取 LSM-Tree 运行的细粒度 trace 数据（如 MemTable 写入节奏、SSTable flush 触发时机、compaction 事件详情、各级存储的 I/O 明细等）存在技术与成本壁垒，这直接限制了不同优化策略的效果对比与验证效率。因此，设计轻量级的 trace 采集机制，并结合可视化技术直观呈现负载特征与策略影响，具有明确的应用价值，也是本课题可重点发力的方向之一。

此外，参数空间的复杂性与可解释性差的问题，也为用户理解与应用 LSM-Tree 优化带来了阻碍。无论是 Bloom 过滤器的内存分配、层级大小的设置，还是 compaction 策略的选择，各类参数不仅数量繁多，且存在复杂的相互影响关系。尽管已有 Monkey 等工作提供了理论层面的最优参数模型，但普通用户与学生仍难以直观感知这些参数在真实系统中的实际效果，而通过可视化界面支持交互式参数调整与实时实验，正是降低理解门槛、提升参数可解释性的有效路径，这也与本课题的设计目标高度契合。

同时，新技术验证与对比缺乏易用平台的问题，也在当前研究中逐渐凸显。随着 learned index（如 Bourbon）、KV 分离（如 WiscKey）、碎片化优化（如 PebblesDB）等创新方案不断涌现，研究社区亟需一个统一、易用的平台，支持快速切换不同策略、观察细粒度性能指标并复现实验结果。而本课题所构建的演示系统，恰好可承担这一角色，为新技术的对比验证提供便捷工具。

综上，国内外研究已在 LSM-Tree 的理论、工程实现与前沿技术探索（如 learned index）上形成丰富成果，但在“教学友好型、交互式操作、细粒度可观测、一键复现实验”的工具层面仍存在明显缺口。本课题通过实现 LSM-Tree 可视化与交互式模拟系统，能够将学术研究中的关键结论——包括 compaction 策略差异、Bloom 过滤器分配效果、KV 分离对性能的影响等——以直观、可调、可对比的方式呈现给使用者，最终同时服务于教学场景的知识传递与研究、工程场景的策略验证。

3.主要研究内容

3.1 LSM-Tree 核心机制实现

本课题的系统实现将围绕基础结构构建、多策略支持与可扩展设计三个维度展开，确保核心功能完备性与未来扩展性的平衡。

在基础结构搭建层面，后端系统将完整实现 LSM-Tree 的核心数据结构与机制：内存层面采用跳表（SkipList）或平衡树实现 MemTable，作为写入操作的临时缓存，确保高效的内存数据插入与查询；持久化存储则通过 SSTable（排序字符串表）实现，包含键值数据块、索引块与过滤块等结构化组件，保障磁盘数据的有序存储与快速定位；同时引入 Write-Ahead Log（WAL）机制，通过日志预写确保写入操作的持久性，避免内存数据丢失；为加速点查（point lookup）过程，系统将集成 Bloom Filter，通过预先过滤不存在的查询请求减少不必要的磁盘 I/O；此外，核心的 Compaction 机制将负责多层 SSTable 的合并与重写，通过维护数据有序性与层级结构平衡，保障系统长期运行的查询效率。这些基础结构的实现将为后续功能扩展提供稳定的底层支撑。

在多策略支持方面，系统将重点集成两种主流 compaction 策略：Size-Tiered Compaction（STC）与 Leveled Compaction（LCS）。其中，STC 通过将相似大小的 SSTable 文件批量合并，可有效降低写入操作的开销；LCS 则通过严格控制每层数据容量上限，减少查询时需要扫描的文件数量，从而降低读放大。为支持策略对比与教学演示，系统将允许用户在可视化界面中动态切换两种策略，并实时观察写放大、延迟、I/O 次数等性能指标的变化，直观呈现不同策略的适用场景与性能差异。

为满足教学场景的多样性与研究需求的扩展性，系统将采用模块化接口设计。通过将数据结构、策略逻辑与性能监控等功能封装为独立模块，后续可便捷引入 LSM-Tree 的各类变体方案——例如 WiscKey 的 Key/Value 分离架构、Monkey 的 Bloom 过滤器动态分配策略等，无需大规模重构底层代码。这种设计既为当前教学提供了基础功能，也为未来纳入前沿研究成果预留了扩展空间，使其能持续服务于更广泛的教学与研究场景。

3.2 性能采集与事件追踪机制

为让用户直观感知 LSM-Tree 运行状态与性能特征，系统将围绕“指标采集 - 事件可视化 - Trace 复用”构建完整技术链路，确保数据实时性、可读性与可复用性。

1. 性能指标体系构建

系统将设计轻量化性能监控与采集框架，以低资源开销实现全维度状态感知。该框架将重点采集四类核心指标：一是资源占用类指标，包括 MemTable 与各层级 SSTable 的内存占用、SSTable 总数量及各层级分布情况，反映系统基础资源消耗；二是性能放大类指标，涵盖写放大（Write Amplification）与读放大（Read Amplification），直接体现不同策略对存储效率的影响；三是 I/O 特征类指标，包含 I/O 操作总次数、单次 I/O 延迟分布，量化磁盘交互效率；四是核心流程类指标，如 Compaction 触发频率、并发线程数量，刻画系统关键操作的动态特征。通过多维度指标联动，为用户提供全面的性能分析依据。

2. 事件追踪与可视化数据流

为将抽象运行过程转化为直观动态效果，系统将对 LSM-Tree 关键事件进行全生命周期追踪与可视化转化。具体包括 MemTable 触发 flush 的完整过程、Compaction 的启动与结束节点、SSTable 文件的生成与删除操作，以及每一次 I/O 交互事件，这些事件将被统一封装为结构化的可视化事件流。同时，系统将通过 WebSocket 建立后端与前端的实时通信通道，确保事件数据能在 1 秒内推送至前端界面并完成渲染，让用户实时观察系统运行的动态变化，避免数据延迟影响认知。

3. 轻量化 Trace 机制

为兼顾实时可视化与后续科研复用需求，系统将构建自定义轻量化 Trace 机制。该机制采用 JSON 或 CSV 格式存储 Trace 数据——两种格式均具备通用性强、易解析的特点，既能直接供前端调用以支撑实时渲染，也可支持用户导出完整 Trace 文件。导出的文件可用于后续科研分析，例如复现实验过程、验证优化策略效果，或与其他系统的 Trace 数据进行对比，进一步拓展系统在研究场景中的实用价值。

3.3 前端交互式可视化展示

为适配教学与研究双场景的直观性需求，系统前端将围绕“动态呈现 - 实时监控 - 交互操作 - 场景适配”四大核心目标，设计多层次可视化功能，让抽象的 LSM-Tree 运行逻辑与性能特征可感、可查、可交互。

1. 动态层次展示

为直观呈现 LSM-Tree 各组件的数据流转逻辑，系统将通过树状层级图与时间轴动画结合的方式，全程可视化关键过程。具体包括：MemTable 接收写入请求后的动态缓存过程，以及数据溢出至磁盘的触发节点；SSTable 从生成、分层存储到层级迁移的完整状态变化；Compaction 机制的触发条件、文件合并流程与合并后的层级更新效果；同时同步展示 BloomFilter 的实时命中率变化，以及读写请求在各层级的访问分布情况。通过动画化的过程呈现，用户可清晰追踪数据从内存到磁盘的全生命周期流转路径。

2. 实时性能监控面板

为帮助用户实时掌握系统运行状态与性能瓶颈，前端将设计多维度实时性能监控面板，采用折线图、柱状图等直观图表类型动态展示核心指标。面板将重点呈现三类数据：一是写入与 I/O 相关指标，包括写入速率波动、I/O 操作总次数与单次延迟分布、写放大系数变化；二是 SSTable 层级指标，如各层级 SSTable 的数量增减与单文件大小分布；三是资源占用指标，涵盖内存（含 MemTable 与 BloomFilter）与磁盘的实时占用比例。所有指标将随系统运行动态更新，支持用户按需切换查看维度，快速定位性能关键影响因素。

3. 交互式控制与策略切换

为用户深度理解参数调整与系统性能的关联，前端将提供灵活的交互式控制功能。用户可在界面上直接调整 LSM-Tree 的核心参数，包括各层级容量比例、Compaction 触发阈值、Bloom Filter 内存分配大小等；参数调整后，系统将实时反馈效果——通过动态更新可视化动画与性能面板数据，直观展示参数变化对数据流转、Compaction 频率、I/O 开销等的影响。这种“调整 - 反馈”的实时交互模式，能有效降低参数空间的理解门槛，帮助用户建立“参数设置 - 性能表现”的直接认知。

3.4 系统封装与部署

在部署层面，系统采用 Docker Compose 实现全服务容器化集成，将前端应用、后端服务与数据库组件统一封装为容器集群。通过编写标准化的配置文件，用户仅需执行简单命令即可完成“一键运行”，无需手动配置依赖环境；同时，容器化设计确保系统能在 Windows、Linux、macOS 等主流操作系统中稳定运行，有效降低跨平台部署的技术门槛，满足教学机、研究服务器、个人设备等多场景的使用需求。

4. 研究方案

4.1 系统架构设计

系统整体采用前后端分离架构，并以可视化驱动的数据采集为核心链路，通过四层结构实现功能解耦与高效协作，确保模拟精度、数据实时性与交互流畅性的平衡。

最底层为存储模拟层（Storage Simulation Layer），是整个系统的核心引擎。该层完整实现 LSM-Tree 的核心模型，包含 MemTable 的内存缓存逻辑、SSTable 的分层存储与管理机制、Write-Ahead Log（WAL）的持久化保障组件，以及 Compaction 的合并调度模块；同时支持高度可配置的运行参数，如层级数量、单文件大小阈值、合并策略选择等，可灵活模拟不同场景下的 LSM-Tree 行为。为支撑上层可视化需求，该层设计了事件回调接口，能将每一次关键操作（如写入、flush、合并）封装为标准化的 trace event，为数据采集提供原始输入。

存储模拟层之上是数据采集与事件追踪层（Tracing & Metrics Layer），承担“数据桥梁”角色。其核心功能包括：实时捕获系统运行时的各类关键事件——如 MemTable 触发 flush、Compaction 启动与结束、读写请求的访问路径等；通过轻量级内存计数器与日志记录机制，同步采集性能指标（如 I/O 操作次数、单次请求延迟、写放大系数等）；最终将事件与指标数据统一封装为 JSON 格式，既满足前端实时可视化的低延迟需求，也为后续科研场景的离线数据分析提供结构化数据支持。

中间层为后端服务层（Backend Service Layer），负责业务逻辑调度与跨层通信。该层拟采用 Python（FastAPI）或 Node.js（Express）开发，核心职责包括：驱动存储模拟层的运行逻辑，响应前端的模拟控制请求；提供 RESTful API 接口，支持用户通过前端界面配置 LSM-Tree 参数（如层级策略、资源阈值等）；通过 WebSocket 建立长连接通道，将数据采集层处理后的实时事件与指标推送至前端；同时负责实验结果与性能日志的持久化存储，支持历史数据回溯。为提升部署灵活性，该层支持 Docker 容器化封装，可实现“一键运行”，并通过多用户隔离机制保障并发实验的稳定性。

最上层为前端可视化与交互层（Frontend Visualization Layer），是用户直接交互的入口。基于 Vue3 或 React 框架开发，该层通过调用后端 API 获取配置权限、接收 WebSocket 推送的实时数据，借助 D3.js 或 ECharts 等可视化工具，动态呈现 LSM-Tree 的层级结构变化、Compaction 的合并动画、性能指标的实时曲线（如写放大趋势、I/O 延迟分布）；同时提供直观的交互控件，支持用户调整策略参数、手动触发模拟流程、导出实验结果数据，最终实现“操作 - 反馈 - 理解”的闭环体验，适配教学与研究的多样化需求。

这种四层架构通过明确的职责划分，既保证了底层模拟的准确性，又实现了数据流转的高效性与前端交互的灵活性，为系统功能的落地提供了稳定的技术支撑。

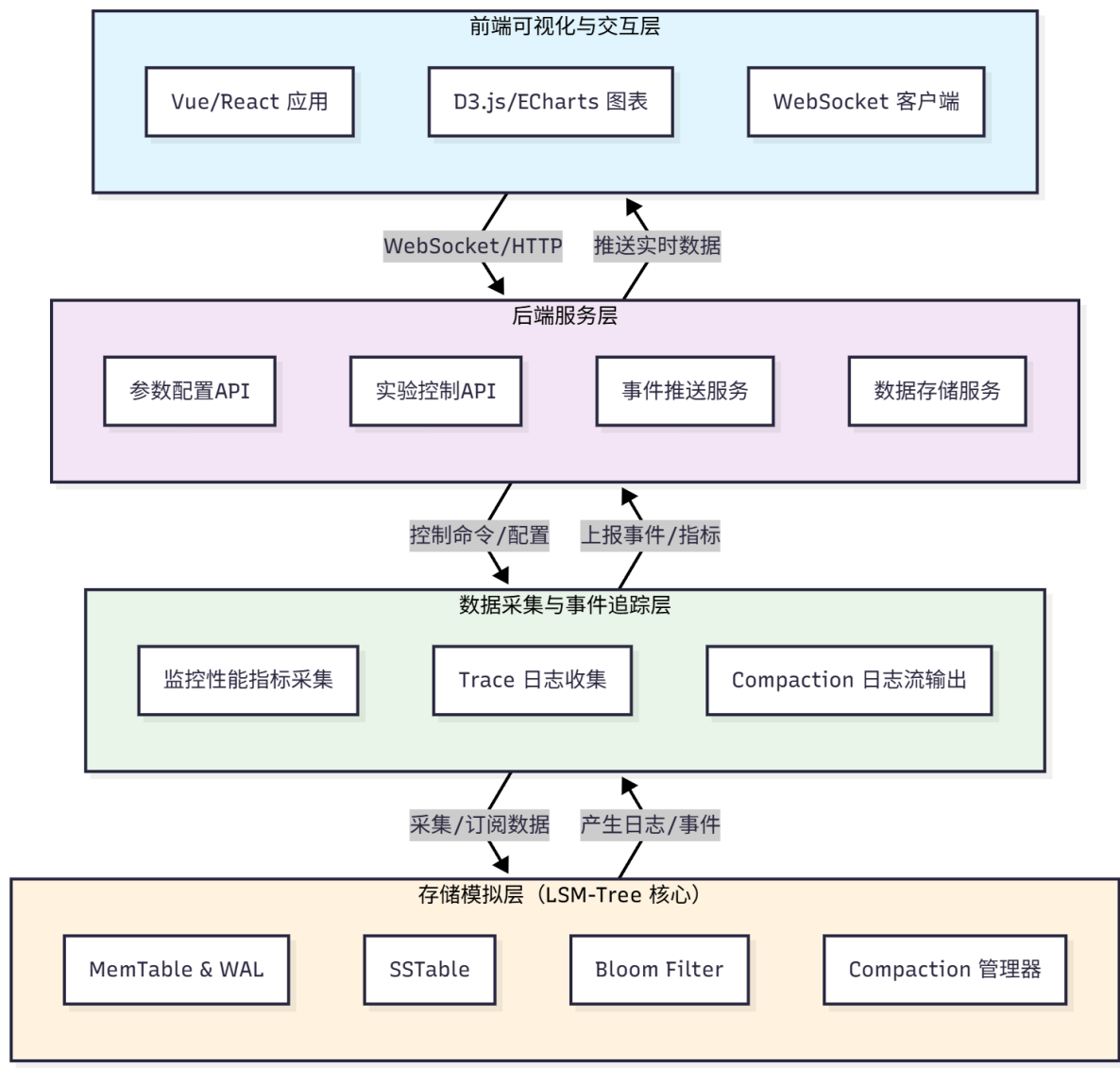


图 1 系统整体架构

5.进度安排和预期达到的目标

5.1 进度安排

毕业设计（论文）各阶段内容	时间安排
掌握 LSM-Tree 的核心原理, 完成系统总体设计与模块划分, 为后续开发奠定基础。	2025 年 10 月 20 日 – 2025 年 11 月 30 日
构建可运行的 LSM-Tree 原型系统, 实现后端核心功能与性能追踪机制。	2025 年 12 月 1 日 – 2025 年 12 月 31 日

实现系统的交互式可视化界面，达到动态演示与实时反馈效果。	2026 年 1 月 1 日 – 2026 年 2 月 15 日
系统集成与容器化部署	2026 年 2 月 16 日 – 2026 年 3 月 31 日
获得实验数据与分析结果，论文结构与系统结果基本完善。	2026 年 4 月 1 日 – 2026 年 5 月 15 日
论文定稿与答辩准备	2026 年 5 月 16 日 – 2026 年 5 月 29 日

5.2 预期目标

本研究旨在构建一套功能完备且可稳定运行的交互式 LSM-Tree 可视化系统，通过直观的交互形式呈现 LSM-Tree 的核心工作机制，为该技术的教学演示、科研探索以及性能分析提供高效便捷的可视化平台。同时，基于系统的功能特性与实验需求，设计并完善可复现多种策略对比的标准化实验报告模板，为后续相关技术的实验验证与成果呈现提供规范参考。

6.课题已具备和所需的条件、经费

本课题以软件系统设计与性能分析为主要研究方向，开发与实验环境需求相对可控。目前，课题已具备较为完备的研究与实现条件。开发工作可在个人笔记本电脑上独立完成。

7.研究过程中可能遇到的困难和问题，解决的措施

在本课题的研究与实现过程中，可能会遇到若干技术与实践层面的困难。首先，LSM-Tree 本身是一种结构复杂、层次众多的存储体系，其核心操作（如 Compaction、Flush、Level 管理等）涉及大量底层 I/O、缓存与线程调度机制。为了在系统中真实还原其运行逻辑，同时保持演示系统的可视化流畅性，需要在“真实性”与“演示性”之间取得平衡。例如，过度追求精确的 I/O 模拟可能导致前端渲染延迟和系统卡顿，而过度简化则可能丢失关键性能特征。因此，需要通过模块化设计和异步架构，将底层逻辑与可视化呈现进行分层解耦，确保既能反映真实运行原理，又能维持交互体验的平滑性。

其次，在性能追踪与事件可视化方面，如何在不显著增加系统开销的前提下实现实时数据采集，是一个较为棘手的问题。由于 LSM-Tree 的写入与合并操作可能高频触发，事件流（Trace）生成若过于冗长，会对后端运行造成干扰，甚至影响演示的连贯性。对此，本课题计划通过自定义轻量化 Trace 机制，仅保留关键事件节点与必要元信息，并在数据采集阶段引入异步缓冲与批量推送策略，从而兼顾实时性与性能开销。此外，还需在可视化层面合理控制事件密度，通过聚合或抽象显示的方式减少前端绘制压力，保证在 1 秒内完成界面更新。

再者，系统架构上可能面临的另一个挑战是前后端通信与同步一致性问题。由于可视化动画与后端事件是异步进行的，若时序管理不当，可能出现“画面提前”或“延迟显示”的现象，影响教学与演示效果。为解决这一问题，系统将采用基于时间戳的事件流排

序机制，并在前端建立统一的渲染队列，通过帧调度（Frame Scheduling）与状态缓存机制实现动画与数据更新的同步。同时，在多策略切换与参数调节场景下，系统还需具备动态重置与重放功能，以支持用户多次交互测试的需求。

此外，研究过程中还可能遇到不同硬件环境下的性能差异与容器化兼容性问题。由于演示系统将通过 Docker 实现跨平台部署，不同主机的资源限制、浏览器性能以及网络延迟都可能对展示效果造成影响。对此，将在设计阶段对资源使用进行合理约束，通过参数化配置文件实现灵活的性能调优，并在 Docker 镜像中预置最小依赖环境，以提升系统的可移植性与稳定性。

8.主要参考文献

- [1] Luo C, Carey M J. LSM-based storage techniques: a survey[J]. The VLDB Journal, 2020, 29(1): 393-418.
- [2] O'Neil P, Cheng E, Gawlick D, et al. The log-structured merge-tree (LSM-tree)[J]. Acta informatica, 1996, 33(4): 351-385.
- [3] Chang F, Dean J, Ghemawat S, et al. Bigtable: A distributed storage system for structured data[J]. ACM Transactions on Computer Systems (TOCS), 2008, 26(2): 1-26.
- [4] Dayan N, Athanassoulis M, Idreos S. Monkey: Optimal navigable key-value store[C]//Proceedings of the 2017 ACM International Conference on Management of Data. 2017: 79-94.
- [5] Broder A, Mitzenmacher M. Network applications of bloom filters: A survey[J]. Internet mathematics, 2004, 1(4): 485-509.
- [6] Huynh A, Chaudhari H A, Terzi E, et al. Towards flexibility and robustness of LSM trees[J]. The VLDB Journal, 2024, 33(4): 1105-1128.
- [7] Yao T, Tan Z, Wan J, et al. SEALDB: An efficient LSM-tree based KV store on SMR drives with sets and dynamic bands[J]. IEEE Transactions on Parallel and Distributed Systems, 2019, 30(11): 2595-2607.
- [8] Dai Y, Xu Y, Ganesan A, et al. From {WiscKey} to bourbon: A learned index for {Log-Structured} merge trees[C]//14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20). 2020: 155-171.
- [9] Cao Z, Dong S, Vemuri S, et al. Characterizing, modeling, and benchmarking {RocksDB}{Key-Value} workloads at facebook[C]//18th USENIX Conference on File and Storage Technologies (FAST 20). 2020: 209-223.

- [10] Dong S, Kryczka A, Jin Y, et al. Rocksdb: Evolution of development priorities in a key-value store serving large-scale applications[J]. ACM Transactions on Storage (TOS), 2021, 17(4): 1-32.
- [11] Lu L, Pillai T S, Gopalakrishnan H, et al. Wiskey: Separating keys from values in ssd-conscious storage[J]. ACM Transactions On Storage (TOS), 2017, 13(1): 1-28.
- [12] Raju P, Kadekodi R, Chidambaram V, et al. Pebblesdb: Building key-value stores using fragmented log-structured merge trees[C]//Proceedings of the 26th Symposium on Operating Systems Principles. 2017: 497-514.
- [13] Ren K, Zheng Q, Arulraj J, et al. SlimDB: A space-efficient key-value storage engine for semi-sorted data[J]. Proceedings of the VLDB Endowment, 2017, 10(13): 2037-2048.
- [14] Yang Zhenkun, Yang Chuanhui, Han Fusheng, Wang Guoping, Yang Zhifeng, Cheng Xiaojun. Architecture and Technology of OceanBase Distributed Relational Database[J]. Journal of Computer Research and Development, 2024, 61(3): 540-554. DOI: 10.7544/issn1000-1239.202330835
- [15] Yang Y U, Huiqi H U, Xuan Z. Optimization of LSM-tree storage systems based on non-volatile memory[J]. Journal of East China Normal University (Natural Science), 2021, 2021(5): 37.